★ Claude Code 실습 · 업무 자동화

카톡 업무 AI 비서 만들기

Claude Code로 읽기, 판정, 대조, 실행을 차례로 구현한다

초보자 단계별 실습 강의 · 2026년 10월 도구 기준

업무 카톡방을 AI가 읽고, 메시지마다 의도를 판정하고, 고객 데이터와 대조해 실수를 잡아내는 비서를 만든다.
모든 실습은 Claude Code에 프롬프트를 붙여 넣고 결과를 확인하는 방식으로 진행한다.

이 강의는 세 가지를 전한다

2026년 10월 16일 공개된 영상 사례를 Claude Code 실습으로 다시 만든다

1

AI는 도구를 붙일수록 할 수 있는 일이 커진다

카톡 읽기에 판정, 대조, 실행을 하나씩 붙이면 보조를 넘어 일을 처리하는 비서가 된다.

2

판정은 작은 모델이, 판단은 큰 모델이 맡는다

수천 건의 라벨 붙이기는 빠르고 싼 모델에 맡기고, 요약과 우선순위는 Claude가 정한다.

3

읽기는 자유롭게, 쓰기는 사람이 승인한 뒤에 한다

메시지 발송과 세금계산서 발행은 되돌리기 어렵다. 규칙 문서로 승인 경계를 먼저 정한다.

목표: 수강생은 프롬프트를 붙여 넣고 확인만 한다. 나머지는 Claude Code가 한다.

카톡 누락이 실제 손실로 이어졌다

영상 제작자가 가족 회사에서 4년간 겪은 '카톡 지옥'에서 출발한 사례다

224건 6초

세 개 방 메시지를 판정하는 데 걸린 시간

1~2원

메시지 판정 비용(영상 속 진술)

22초

카톡방을 읽고 사진 여섯 장까지 확인한 시간

월요일

목요일에야 알았을 실수를 당일에 발견했다

영상이 보여 준 것

- AI가 카톡을 직접 읽고 판정한다
- ERP와 대조하고 발행을 요청한다

이 강의에서 더하는 것

- 연습 데이터로 안전하게 따라 한다
- 모든 단계를 Claude Code로 한다

비서는 네 층으로 이루어진다

읽기, 판정, 대조, 실행을 Claude Code가 지휘하고 규칙 문서가 기준을 준다

Claude Code가 네 단계를 차례로 지휘한다

① 읽기
카톡 메시지를
JSON으로 가져온다
(kakaocli)

→

② 판정
메시지마다
의도 라벨을 붙인다
(Haiku 또는 Jev)

→

③ 대조
고객 데이터와
ERP 화면에 맞춘다
(Claude in Chrome)

→

④ 실행
사람이 승인하면
보내고 요청한다
(볼타 API)

규칙 문서가 AI의 판단 기준이 된다

CLAUDE.md, labeling_guide.md, action_policy.md

영상의 장면은 강의의 단계와 대응한다

괄호 안은 영상 속 시각이다. 0단계는 영상 마지막 장면에 해당한다

영상 장면	강의 단계	핵심 기술
카톡 대화 읽고 판단하기 (01:19)	1단계, 2단계	kakaocli, 의도 판정
한 달치 카톡 분석 (03:18)	2단계	대량 일괄 판정
일정 관리 프로그램 연결 (05:38)	3단계	Claude in Chrome
세금계산서 발행 요청 (07:12)	4단계	브라우저 입력, 볼타 API
실제 실수 사례 (09:02)	5단계	확정값 장부 대조
컴퓨터 사용 시작하는 법 (11:54)	0단계	앱 설정에서 기능 켜기

00

0단계 준비

기본 지식과 작업 환경

Claude Code, 접근성 API, 판정 모델이 무엇인지 먼저 이해하고 작업 폴더를 만든다

Claude Code는 말로 일을 맡기는 AI다

터미널(Terminal)에서 대화하며 파일을 만들고 프로그램을 실행하는 에이전트형 도구다

수강생은 프롬프트를 붙여 넣고 결과를 확인한다.
설치, 실행, 오류 해결은 Claude Code가 한다.

Claude Code가 하는 일

- 폴더와 파일을 만든다
- 프로그램을 설치하고 실행한다
- 오류를 읽고 스스로 고친다

사람이 하는 일

- 원하는 결과를 말로 설명한다
- 권한 요청을 읽고 승인한다
- 결과가 맞는지 확인한다

카톡에는 개인 채팅을 읽는 공식 API가 없다

그래서 아래 세 가지 우회 경로 중 하나를 쓴다

방식	원리	속도	이 강의
컴퓨터 사용 (Computer Use)	화면을 찍어 AI가 글자를 읽는다	느림	영상 초반에 시도
접근성 API (손쉬운 사용)	화면낭독용 통로로 글자와 버튼을 읽는다	빠름	영상의 핵심 방식
로컬 DB 읽기	카톡 Mac 앱이 저장한 DB를 읽기 전용으로 조회한다	가장 빠름	kakaocli의 읽기 방식

접근성 API는 시각장애인을 위한 기능으로, 앱이 화면에 어떤 글자와 버튼이 있는지 알려 준다

메시지 판정은 작고 빠른 모델에 맡긴다

큰 모델은 요약과 우선순위를, 작은 모델은 라벨 붙이기를 맡는다

구분	Claude Haiku	Jev (TypeSafe AI)
무엇인가	Anthropic의 소형 모델	선택지와 확률만 돌려주는 판정 전용 모델 (2026년 9월 공개)
준비물	없음. Claude 구독으로 바로 쓴다	별도 가입과 API 키
답의 형식	글로 답하므로 형식 검사가 필요하다	정해 둔 선택지 안에서만 답한다
속도와 비용	빠르고 저렴하다	70~500ms, 입력 100만 토큰당 0.042달러
이 강의	기본 경로	선택 실습

실습에 쓰는 도구는 다섯 가지다

영상의 Orca는 여러 에이전트를 나란히 돌리는 앱이다. 이 강의에서는 쓰지 않는다

도구	역할	언제 필요한가
Claude Code	전체를 지휘하고 코드를 만들어 실행한다	모든 단계
kakaocli	카톡 메시지를 JSON으로 꺼낸다	실제 카톡 트랙
Claude Haiku 또는 Jev	메시지마다 의도 라벨을 붙인다	2단계 (Haiku는 기본 포함)
Claude in Chrome	로그인된 ERP 화면을 읽고 입력한다	3단계, 4단계
볼타(Bolta) API	전자세금계산서를 발행한다	4단계 선택 실습

권한 두 가지와 예비 계정을 준비한다

권한 설정만은 사람이 시스템 설정에서 직접 한다

Claude 구독

Pro 또는 Max

Claude Code를 설치하고 /login으로 로그인한다

손쉬운 사용

시스템 설정

개인정보 보호 및 보안에서 터미널을 켠다

전체 디스크 접근

시스템 설정

카톡 DB를 읽으려면 함께 켠다

예비 계정

테스트용 단톡방

실제 업무방 대신 연습용 방에서 먼저 한다

카카오톡 자동화는 비공식 방식이다

- 자동화 도구 사용은 계정 이용 제한 대상이 될 수 있다
- 대량 발송이나 무인 자동응답으로 넓히지 않는다

컴퓨터 사용 기능은 앱 설정에서 켠다

영상 마지막 장면(11:54)을 Claude 앱과 Claude Code 기준으로 정리했다

Claude 데스크톱 앱

- 설정에서 데스크톱 앱 항목을 연다
- 컴퓨터 사용 스위치를 켠다
- 앱마다 처음에 승인 창이 뜬다

Claude Code

- /mcp에서 computer-use를 켠다
- 손쉬운 사용과 화면 기록을 허용한다
- 멈추려면 Esc를 누른다

메뉴 위치는 버전마다 조금 다르다. 카톡 보내기는 4단계 선택 실습에서 해 본다

카톡은 로그인만 돼 있으면 Claude가 연다

읽기 도구는 창 없이도 되지만, 화면 조작과 보내기는 로그인된 카톡이 필요하다

자동 로그인을 켠 경우

- 실행하면 바로 채팅 목록이 뜬다
- 무인 예약 실행까지 돌릴 수 있다
- PC를 켜는 누구나 내 카톡에 들어간다

직접 로그인해 켜 둔 경우

- 작업 전에 본인이 로그인한다
- 창을 닫아도 트레이에 있으면 유지된다
- 종료나 재부팅 뒤에는 다시 로그인한다

확인법: 카톡을 완전히 종료하고 다시 실행했을 때 바로 채팅 목록이 뜨면 자동 로그인이다

로그인 화면이 뜨면 Claude는 비밀번호를 넣지 않고 멈춘다. 카톡 잠금 모드도 확인한다

첫 프롬프트로 폴더와 안전 규칙을 만든다

CLAUDE.md는 Claude Code가 작업할 때마다 먼저 읽는 프로젝트 규칙 파일이다

[입력] 프롬프트 · Claude Code

```
~/kakao-secretary 폴더를 만들고
앞으로 그 안에서 작업해 줘.

1) Homebrew, uv, Xcode 도구가 설치돼
   있는지 확인하고, 없으면 설치 방법을
   알려 줘. 설치는 내 승인 후에 해.

2) CLAUDE.md를 만들고 아래 규칙을 적어 줘.
   - 카톡 메시지는 읽기만 한다
   - 보내기와 발행은 내가 '보내'라고
     말할 때만 한다
   - API 키 값은 화면에 출력하지 않는다
   - 결과는 data, reports 폴더에 둔다
```

[출력] Claude가 생성 · 폴더 구조

```
kakao-secretary/
 ├── CLAUDE.md        안전 규칙
 ├── data/raw/        카톡 원본
 ├── data/labeled/    판정 결과
 ├── rules/           판정과 실행 규칙
 ├── context/         ERP 안내 문서
 ├── reports/         매일 보고서
 └── logs/            실행 기록
```

빈 폴더는 다음 단계에서 채운다.

01

1단계 읽기

카톡 메시지를 텍스트로 꺼낸다

연습 데이터로 먼저 익히고, 준비가 되면 실제 카톡으로 옮긴다

연습 데이터로 먼저 익히고 실제 카톡은 나중에

두 트랙은 같은 JSON 형식을 쓰므로 2단계부터는 똑같이 진행한다

A 트랙: 연습 데이터

- Claude Code가 가짜 카톡을 만든다
- 계정 위험과 개인정보 걱정이 없다
- 강의실에서 모두 같은 결과를 본다

B 트랙: 실제 카톡

- kakaocli로 내 카톡을 읽는다
- 예비 계정과 테스트방에서 한다
- Mac과 권한 설정이 필요하다

권장 순서: A 트랙으로 6단계까지 완주한 뒤 B 트랙으로 바꾼다

실습 1-A: 연습용 카톡 대화를 만든다

영상의 실수 사례를 그대로 담은 고객방과 공장방을 Claude Code가 만든다

[입력] 프롬프트 · Claude Code

```
data/raw 폴더에 연습용 카톡 대화를
JSON으로 만들어 줘.
- 형식은 kakaocli와 같게: author,
  message, sentAt
- 방 3개: 고객방_A사, 공장방, 사무실방
- 방마다 메시지 30개, 최근 2주 날짜
- 고객방 흐름: 120칸 금요일 가능? →
  가능합니다 → 150칸 변경 → 140칸 금요일 오전
- 공장방 흐름: 월요일 가능 →
  내일 조립, 120칸 맞죠?
- 세금계산서 발행 요청 1건, 잡담 다수
- data/customers.csv(거래처 3곳)와
  data/erp_mock.csv(설치 일정)도 만들어 줘
끝나면 방별 메시지 수를 표로 보여 줘.
```

[출력] Claude가 생성 · data/raw/고객방_A사.json

```
[
  {"author": "A사 김과장",
   "sentAt": "2026-10-19 10:12",
   "message": "120칸 금요일까지 되나요?"},
  {"author": "나",
   "sentAt": "2026-10-19 10:15",
   "message": "네, 가능합니다"},
  ...
]
```

author는 보낸 사람, sentAt은 보낸 시각,
message는 본문이다.

실습 1-B: kakaocli로 실제 카톡을 읽는다

예비 계정의 테스트방에서만 한다. 보내기 기능은 아직 쓰지 않는다

[입력] 프롬프트 · Claude Code

```
kakaocli를 설치하고 카톡을 읽기 전용으로
점검해 줘.

1) brew로 kakaocli를 설치해
2) kakaocli status로 권한을 확인해
3) kakaocli auth로 DB 복호화를 확인해
4) 채팅방 목록 10개를 보여 줘
5) '테스트방'의 최근 7일 메시지를
   data/raw/테스트방.json에 저장해

send 명령은 절대 실행하지 마.
오류가 나면 원인과 해결책을 먼저 설명하고
내 승인을 받아.
```

[출력] 결과 · Claude Code가 실행한 명령

```
$ brew install silver-flight-group/
      tap/kakaocli
$ kakaocli status
$ kakaocli auth
$ kakaocli chats --limit 10 --json
$ kakaocli messages --chat "테스트방"
      --since 7d --json
```

이렇게 보이면 성공

- 보낸 사람, 시각, 본문이 보인다

1단계는 권한과 활성 창에서 자주 막힌다

오류 메시지는 그대로 Claude Code에 붙여 넣고 해결을 맡긴다

증상	원인	해결
status에서 권한 오류가 난다	손쉬운 사용이나 전체 디스크 접근이 꺼져 있다	시스템 설정에서 터미널을 켠다
status는 정상인데 auth만 실패한다	사용자 ID를 자동으로 찾는 시간이 부족하다	오류를 붙여 넣고 재시도를 맡긴다
채팅방이 보이지 않는다	카톡이 화면 맨 앞에 없다 (접근성 방식)	카톡을 별도 데스크톱에 띄워 둔다
업데이트 뒤 갑자기 안 된다	비공식 도구라 카톡 변경에 깨진다	도구를 업데이트하고 A 트랙으로 계속한다

02

2단계 판정

메시지마다 의도 라벨을 붙인다

작은 모델이 빠르게 분류하고, Claude가 할 일과 답장 초안을 정리한다

라벨은 여섯 가지로 시작한다

영상에 나온 분류를 업무 카톡에 맞게 정리했다. 질문은 '정보 요청'에 넣는다

라벨	뜻	예
잡담	업무와 관계없다	점심 맛있게 드세요
정보 요청	상대가 정보를 원한다	견적서 다시 보내 주실 수 있나요?
요청	행동을 요구한다	세금계산서 발행 부탁드립니다
결정	합의하거나 확정한다	그럼 140칸으로 갈게요
일정	날짜와 납기를 다룬다	금요일 오전까지 가능할까요?
문제	오류, 불만, 장애를 알린다	저장이 안 돼요

판정 결과는 같은 형식으로 남긴다

형식이 같아야 3단계 대조와 5단계 검증에서 그대로 쓸 수 있다

[출력] 결과 · data/labeled/고객방_A사.json

```
{
  "msg_id": "고객방-0042",
  "label": "결정",
  "entities": {
    "수량": "140칸",
    "날짜": "금요일 오전",
    "거래처": "A사" },
  "needs_action": true,
  "confidence": 0.93
}
```

읽는 법

- label: 여섯 라벨 중 하나다
- entities: 수량, 날짜, 거래처를 뽑아 둔다
- confidence: 0.6 미만은 사람이 다시 본다

실습 2-1: Haiku로 모든 메시지를 판정한다

API 키 없이 Claude 구독만으로 실행하는 기본 경로다

[입력] 프롬프트 · Claude Code

```
data/raw의 카톡 메시지를 한 건씩 판정하는
스크립트를 만들고 실행해 줘.

- 판정은 claude -p --model haiku로 하고
  20건씩 묶어서 보내
- 라벨: 잡담, 정보 요청, 요청, 결정,
  일정, 문제 중 하나
- 수량, 날짜, 거래처, 금액은 entities로
- 기준은 rules/labeling_guide.md를 쓰고,
  없으면 라벨 표로 초안을 만들어
- 결과는 data/labeled/방이름.json
- 마지막에 건수와 걸린 시간을 출력해
실행 후 애매한 판정 5건을 보여 줘.
```

[출력] 결과 예시 · 실행 요약

```
판정 완료: 3개 방, 90건
걸린 시간: 41초

라벨 분포
  잡담 38 | 정보 요청 12 | 요청 9
  결정 14 | 일정 11 | 문제 6

애매한 판정 5건
  #17 "그걸로 가시죠"      결정? 0.55
  #23 "확인 부탁드려요"    요청? 0.58
  ...
```

실습 2-2: 애매한 사례를 기준 문서에 쌓는다

영상의 '우리 기준을 계속 먹여 주면 정확해진다'가 바로 이 작업이다

[입력] 프롬프트 · Claude Code

```
방금 애매했던 판정의 정답을 알려 줄게.
rules/labeling_guide.md에 '애매한 사례'
항목으로 추가하고, 같은 유형이 다시 나오면
이 기준을 따르도록 해 줘.

#17 '그걸로 가시죠'  → 결정
#23 '확인 부탁드려요' → 요청
#41 '내일 뵙겠습니다' → 일정

추가한 뒤 전체 판정을 다시 돌리고
바뀐 건수를 알려 줘.
```

[출력] Claude가 생성 · labeling_guide.md

```
## 애매한 사례
- 앞 메시지의 안을 받아들이는 말
  ('그걸로 가시죠')은 결정이다
- 확인을 부탁하는 말은 행동을
  요구하므로 요청이다
- 만남 약속은 일정이다

다시 판정: 90건 중 4건 변경
```

실습 2-3(선택): Jev로 판정 형식을 고정한다

API 키는 사람이 직접 발급받아 .env 파일에 붙여 넣는다. 채팅창에 키를 쓰지 않는다

[입력] 프롬프트 · Claude Code

```
판정 스크립트에 Jev를 선택지로 추가해 줘.

- .env에 TYPESAFE_API_KEY가 있으면
  Jev(jev-latest)로, 없으면 Haiku로 판정해
- 라벨은 선택형(choice) 질문으로,
  조치 필요 여부는 예/아니오형으로 물어
- 확률 0.6 미만은 '사람 확인'으로 표시해
- Haiku 결과와 Jev 결과가 다른 건을
  표로 비교해 보여 줘
- 키 값은 출력하지 마
```

[출력] Claude가 생성 · Jev 요청 본문

```
{ "model": "jev-latest",
  "state": "A사 김과장: 140칸으로
     바꿀게요. 금요일 오전 맞죠?",
  "questions": {
    "label": { "type": "choice",
      "options": ["잡담","정보 요청",
        "요청","결정","일정","문제"] },
    "action": { "type": "noul",
      "instructions": "조치가 필요한가" }
  } }
```

응답은 선택지마다 확률로 돌아온다.

실습 2-4: 할 일과 답장 초안을 받는다

영상 01:57 장면처럼 우선순위와 보낼 메시지 초안까지 받는다

[입력] 프롬프트 · Claude Code

```
판정 결과를 바탕으로 오늘 아침 보고서를
reports/오늘날짜_brief.md로 만들어 줘.

1) 해야 할 일을 급한 순서로 정리해
2) 할 일마다 근거 메시지를 붙여
3) 상대에게 보낼 답장 초안을 써
4) 답장은 보내지 말고 초안만 둬
```

[출력] 결과 예시 · reports/..._brief.md

```
# 오늘 할 일 (3건)
1. [급함] 사무실방: 저장 오류 보고
   근거: B대리 '저장이 안 돼요'
   초안: 확인 후 오늘 중 고치겠습니다
2. 고객방_A사: 세금계산서 발행 요청
   초안: 오늘 오후 발행 예정입니다
3. 공장방: 조립 수량 확인 필요
   → 5단계 교차 검증에서 다룬다
```

03

3단계 대조

업무 데이터와 맞춰 본다

고객 데이터와 일정 관리 화면을 읽어 카톡 내용이 반영됐는지 확인한다

AI에게 우리 회사의 맥락을 알려 준다

영상에서 'AI가 이 ERP가 무엇인지 알고 있다'고 말한 부분이다

연결 대상	방법	난이도
고객 데이터 (엑셀, CSV)	파일을 data 폴더에 두고 읽는다	쉬움
ERP (API가 있는 경우)	API로 직접 조회한다	보통
ERP (API가 없는 옛 시스템)	Claude in Chrome으로 화면을 읽는다 (영상 방식)	보통

맥락 문서에 적을 것

- 메뉴 위치와 열 이름, 그리고 카톡의 '칸'이 화면의 '수량'이라는 식의 용어 대응을 적는다

실습 3-1: 질문을 받아 ERP 안내서를 만든다

초보자는 문서를 직접 쓰지 않는다. 메타프롬프트로 Claude가 묻고 정리하게 한다

[입력] 프롬프트 · Claude Code

```
context/erp_guide.md를 만들려고 해.
우리 회사 일정 관리 프로그램을 설명할 수
있도록 나에게 질문을 하나씩 해 줘.

- 접속 주소와 로그인 방식
- 확인할 메뉴와 화면의 열 이름
- 카톡 용어와 화면 용어가 다른 경우

답을 다 받으면 문서로 정리해 줘.
(A 트랙은 data/erp_mock.csv를 설명해 줘)
```

[출력] Claude가 생성 · erp_guide.md

```
# 일정 관리 프로그램 안내
- 주소: (사내 주소)
- 로그인: 크롬에 이미 로그인돼 있다
- '설치 일정' 메뉴: 다음 주 설치 건
- 열: 거래처, 제품, 수량, 설치일,
  담당자
- 카톡의 '칸'은 화면의 '수량'이다
```

실습 3-2: ERP를 읽고 카톡과 대조한다

브라우저를 쓰려면 Claude Code를 claude --chrome으로 시작해야 한다

[입력] 명령어 · 터미널에 입력

```
claude --chrome
```

[입력] 프롬프트 · Claude Code

```
context/erp_guide.md를 읽고 크롬으로
'설치 일정' 화면의 다음 주 건을 읽어 줘.
읽기만 하고 아무것도 누르지 마.
(A 트랙은 data/erp_mock.csv를 읽어)
그다음 data/labeled 판정 결과와
data/customers.csv를 대조해서
- 카톡에서 바뀌었는데 ERP에 없는 건
- 일정에 영향을 줄 문제 보고
를 표로 정리해 줘.
```

[출력] 결과 예시 · 대조 표

```
건          카톡          ERP          판정
A사 설치    140칸         120칸        불일치
B사 설치    목요일        목요일       일치
사무실      저장 오류     -            일정 영향

오늘 할 일
1. A사 수량을 ERP에 반영하기 전에
   고객과 공장에 확인한다
2. 저장 오류를 다음 주 설치 전에 고친다
```

04

4단계 실행

승인 경계를 정하고 실행을 붙인다

카톡 답장과 세금계산서 발행 요청은 사람이 승인한 뒤에 한다

읽기는 자유롭게, 쓰기는 승인 뒤에 한다

rules/action_policy.md에 상황별로 AI의 몫과 사람의 몫을 적는다

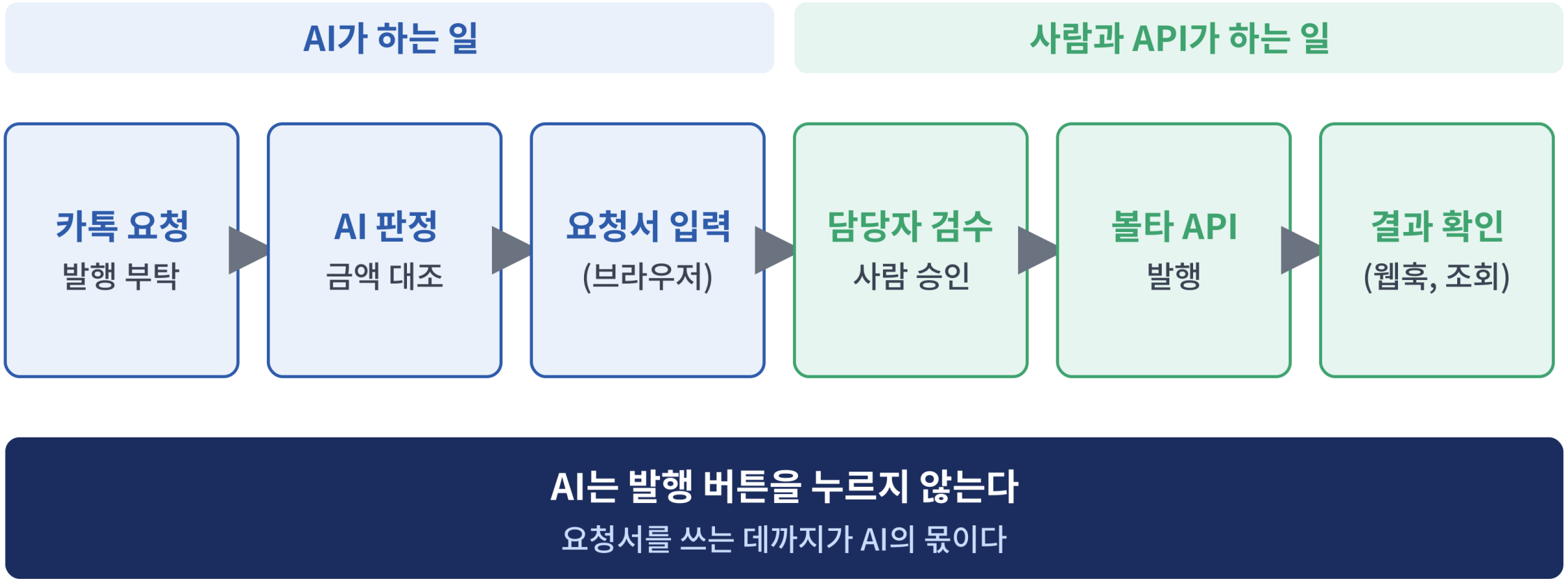
상황	AI가 하는 일	사람 승인
카톡 답장	초안을 쓴다. 선택 실습에서는 전송 직전까지 입력한다	보내기 직전 확인
일정 변경 감지	ERP 수정안과 담당자 알림 초안을 쓴다	필요
세금계산서 요청	요청서 입력까지만 한다	발행은 사람이 한다
수량이나 금액 불일치	즉시 보고하고 책임자를 부른다	사람이 판단한다

영상에서도 실제 발행은 담당자가 검수한 뒤 볼타 API로 넘어간다

FLOW

AI는 요청서까지, 발행은 사람이 한다

영상 07:12 장면의 흐름이다. 볼타(Bolta)는 전자세금계산서 발행 API다



실습 4: 테스트 키로 발행 요청서를 만든다

test_로 시작하는 테스트 키는 유료 구독 없이 발급되며 실제 발행이 되지 않는다

[입력] 프롬프트 · Claude Code

```
판정 결과에서 세금계산서 발행 요청을
찾아 거래처, 금액, 품목, 공급일을 뽑고
data/customers.csv와 대조해 줘.

불일치가 없으면 .env의 BOLTA_TEST_KEY로
발행 요청 JSON을 만들어 보여 줘.
실제 전송은 내가 '보내'라고 할 때만 해.
API 키 값은 출력하지 마.
```

볼타 연습 순서

- 개발자센터에서 test_ 키를 받는다
- API 응답은 접수일 뿐 발행 완료가 아니다
- 최종 결과는 웹훅이나 조회 API로 본다
- 실제 발행에는 공동인증서 등록이 필요하다

컴퓨터 사용으로 카톡을 직접 다룰 수 있다

이름을 검색해 방을 열고 읽고 보내기까지 한다. 카톡은 전체 제어 등급의 앱이다

하려는 일	맞는 방법
특정인에게 가끔 메시지 보내기	컴퓨터 사용 (보내기 직전 사람 확인)
한두 방의 최근 대화 빠르게 확인	컴퓨터 사용
여러 방의 대량 메시지 읽기와 판정	DB 읽기 도구 (kakaocli)
매일 자동으로 도는 보고서	DB 읽기 도구와 예약 실행

브라우저는 보기만, 터미널은 클릭만 허용되지만 카톡 같은 일반 앱은 클릭과 입력이 모두 된다

컴퓨터 사용에는 다섯 가지 제약이 있다

소량의 확인과 보내기에는 편하지만 대량 처리와 무인 실행에는 맞지 않는다

제약	의미
느리다	스크린샷을 찍고 판단하기를 반복해 한 달치 대화 읽기에는 맞지 않는다
무인 실행이 안 된다	claude -p 같은 예약 실행에서는 쓸 수 없다
사람이 있어야 한다	앱 승인 창이 뜨고, PC가 깨어 있어야 하며, 한 번에 한 세션만 쓴다
잘못 보낼 수 있다	이름이 비슷한 방을 고르면 엉뚱한 곳으로 메시지가 간다
Mac에서는 두 곳 모두 된다	Claude Code(/mcp)와 데스크톱 앱 모두 쓸 수 있다

실습 4-2(선택): 전송 직전에 멈추게 한다

Claude Code에서 테스트방으로만 한다. 카톡을 미리 열 필요는 없다

[입력] 프롬프트 · Claude Code (computer-use 켬)

```
카카오톡을 열어 줘. 꺼져 있으면 실행하고,
로그인 화면이 뜨면 멈추고 나에게 알려 줘.

1) 검색에서 '테스트방'을 찾아 열어 줘
2) 최근 메시지 10개를 읽고 요약해 줘
3) 아래 문구를 입력창에 넣어 줘
   '확인 후 오늘 중 고치겠습니다'
4) 전송은 누르지 말고 멈춰서
   받는 방 이름과 문구를 보여 줘
5) 내가 '보내'라고 하면 그때 보내 줘
```

[출력] 결과 예시 · 전송 직전 확인

```
받는 방: 테스트방 (참여자 2명)

최근 메시지 요약
  저장 오류 보고 1건, 일정 문의 1건

입력한 문구
  확인 후 오늘 중 고치겠습니다

전송 버튼은 아직 누르지 않았습니다.
'보내'라고 하시면 전송합니다.
```

05

5단계 교차 검증

방과 방 사이의 실수를 잡는다

영상 제작자가 실제로 겪은 수량과 날짜 착오를 그대로 재현한다

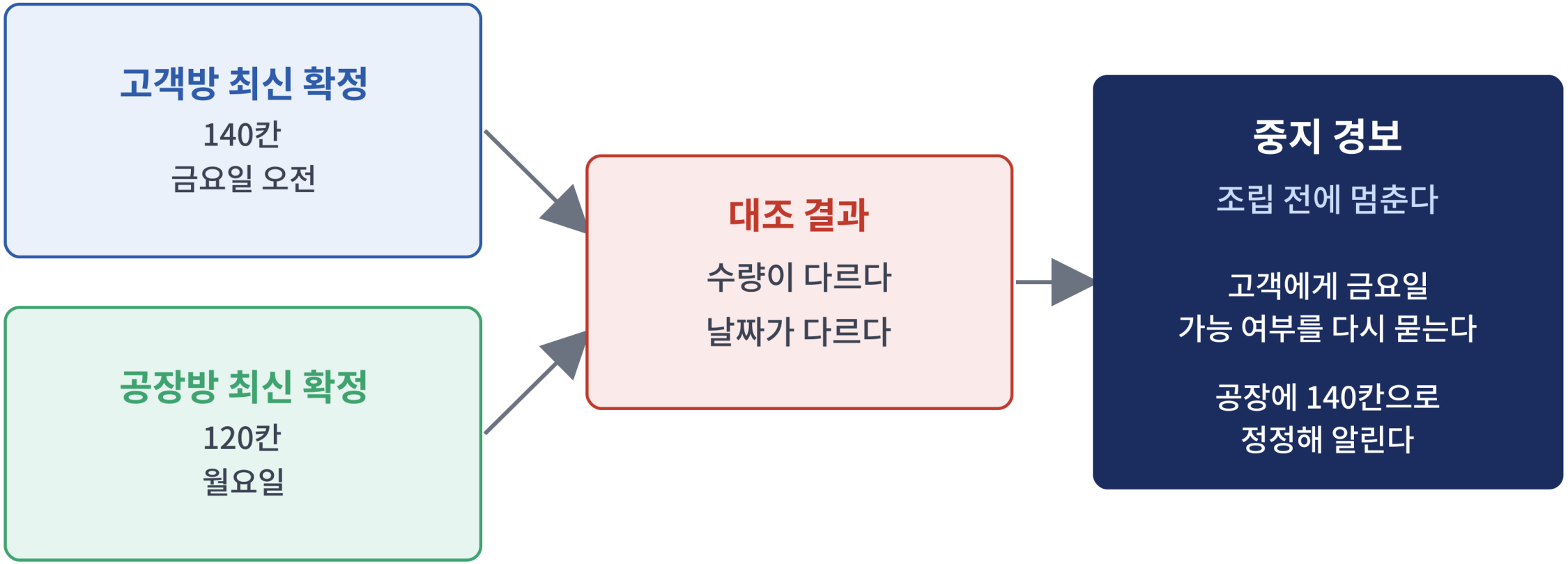
두 방의 확정값이 어긋나 틀린 수량이 확정됐다

영상 09:02 장면. 제작자가 실제로 겪은 일을 조금 바꾼 사례다

순서	고객방	공장방	무엇이 문제인가
1	120칸, 금요일까지 되나요? → 가능합니다		
2		월요일에 가능해요. 월요일로 잡을게요	고객에게 날짜 변경을 안 알렸다
3	150칸으로 늘릴게요 → 140칸, 금요일 오전 맞죠?		공장에 변경을 안 알렸다
4		내일 조립 들어갑니다. 120칸 맞죠? (엄지 표시)	틀린 수량을 확정했다

확정값을 대조하면 조립 전에 멈춘다

사람이라면 목요일에야 알았을 실수를 AI는 월요일 당일에 잡는다



실습 5: 두 방을 대조해 경보를 받는다

같은 건은 '같은 거래처이면서 같은 제품'으로 묶는다

[입력] 프롬프트 · Claude Code

```
고객방_A사와 공장방에서 '결정'과 '일정'
라벨 메시지만 모아 거래처와 제품별
최신 확정값 장부를 만들어 줘.

두 방의 수량이나 날짜가 다르면
'중지 경보'로 표시하고,
고객 확인 메시지와 공장 정정 메시지
초안을 각각 써 줘.
결과는 reports/날짜_mismatch.md에 저장해.
```

[출력] 결과 예시 · ..._mismatch.md

```
## 중지 경보: A사 설치 건
수량  고객방 140칸 | 공장방 120칸
날짜  고객방 금요일 오전 | 공장방 월요일

고객에게:
  금요일 오전 설치가 어려울 수 있어
  확인드립니다. 월요일도 괜찮으실까요?
공장에:
  수량을 140칸으로 정정합니다.
  조립 전에 확인 부탁드립니다.
```

06

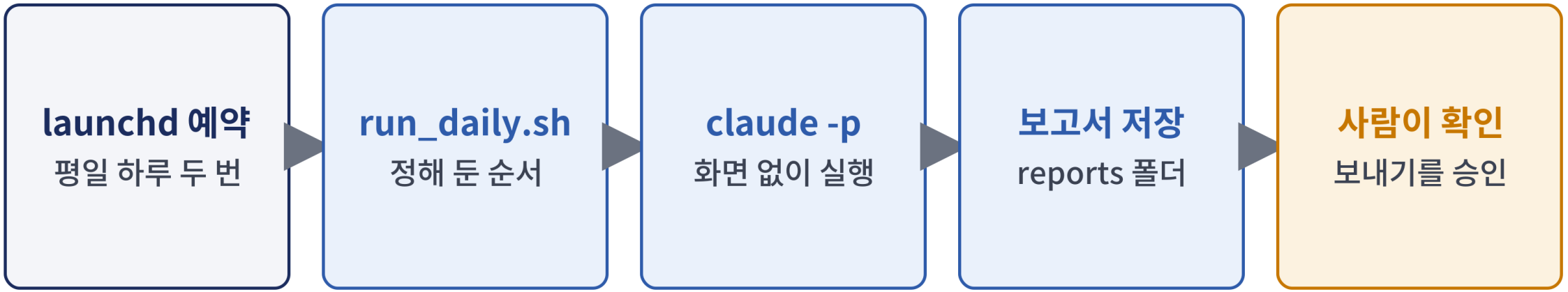
6단계 정기 실행

매일 정해진 시각에 돌게 만든다

Mac의 launchd가 Claude Code를 화면 없이 실행해 보고서를 쌓는다

예약 실행은 보고서까지만 만든다

카톡이 Mac에 있으므로 클라우드가 아니라 Mac에서 예약한다



예약 실행은 읽기와 초안 작성까지만 한다

카톡 읽기, 판정, 고객 CSV 대조, 불일치 검사, 보고서

보내기와 발행은 사람이 보고서를 읽은 뒤에 한다

실습 6: 매일 도는 스크립트를 예약한다

claude -p는 대화창 없이 한 번 실행하고 끝나는 헤드리스(headless) 모드다

[입력] 프롬프트 · Claude Code

```
1~5단계를 묶은 run_daily.sh를 만들어 줘.
순서: 카톡 읽기 → 판정 → 고객 CSV 대조
      → 불일치 검사 → reports에 보고서 저장

- 보내기와 발행은 넣지 말고 초안만 둬
- 판정은 claude -p --model haiku로 해
- 권한 질문은 거절되게 해
  (--permission-prompts none)
- 평일 08:50, 13:50에 실행되는 launchd
  설정을 만들고 등록 방법을 알려 줘
- 실행 기록은 logs 폴더에 남겨
```

예약 실행에서 알아 둘 점

- launchd는 터미널의 로그인 상태를 모른다
- claude setup-token으로 받은 토큰을 쓴다
- 이 토큰으로는 크롬 연동이 꺼진다
- 그래서 ERP 확인은 사람이 있을 때 한다

막히면 상황을 그대로 Claude Code에 알린다

자주 나오는 상황과 그때 할 일이다

상황	할 일
권한 창이 떴는데 무슨 뜻인지 모르겠다	"이 명령이 무엇을 하는지 설명하고 기다려"라고 입력한다
판정이 자꾸 틀린다	정답을 알려 주고 labeling_guide에 추가하게 한다
카톡 로그인 화면에서 멈췄다	본인이 로그인한 뒤 "계속해"라고 입력한다
크롬이 연결되지 않는다	/chrome을 입력해 연결 상태를 보고 다시 연결한다
예약 실행이 돌지 않는다	"logs 폴더의 최근 기록을 읽고 원인을 찾아"라고 한다
카톡 업데이트 뒤 읽기가 멈췄다	kakaocli를 업데이트하게 하고, 그동안 A 트랙으로 진행한다

SAFETY

일곱 가지 안전 원칙은 끝까지 지킨다

실습이 끝나고 실제 업무에 붙일 때 더 중요해진다

계정과 개인정보

- 비공식 자동화는 계정 제한 위험이 있다
- 카톡에는 고객과 직원 정보가 섞여 있다
- 사내 보안 규정을 먼저 확인한다
- 자동 로그인은 개인 PC에서만 켠다

권한과 실행

- 접근성 권한은 다른 앱도 조작한다
- 돈과 외부 발송은 사람이 승인한다
- 업데이트 뒤에는 동작을 다시 본다

이것이 보이면 다음 단계로 넘어간다

강의 중 수강생이 스스로 확인하는 기준이다

단계	확인할 결과
0단계	CLAUDE.md와 작업 폴더가 만들어졌다
1단계	data/raw에 방별 JSON이 있고 보낸 사람, 시각, 본문이 보인다
2단계	모든 메시지에 라벨이 붙고 아침 보고서가 나왔다
3단계	카톡과 ERP(또는 erp_mock)의 대조 표가 나왔다
4단계	발행 요청 JSON이 나왔고, 선택 실습에서는 전송 직전에 멈췄다
5단계	A사 설치 건에 중지 경보가 떴다
6단계	예약한 시각에 reports 폴더에 보고서가 쌓인다

DISCUSSION

현장에 옮기기 전에 네 가지를 묻는다

우리 회사 업무에 적용할 때 팀이 함께 정할 질문이다

Q1	우리 회사 카톡에서 '결정'과 '요청'을 가르는 기준은 무엇인가?	판정 기준
Q2	AI가 사람 승인 없이 해도 되는 일은 어디까지인가?	승인 경계
Q3	카톡 말고 확정값을 기록해 둘 공식 장소가 있는가?	기록 체계
Q4	고객과 직원에게 AI가 메시지를 읽는다는 사실을 어떻게 알리는가?	개인정보

규칙을 문서로 만든 회사가 AI를 잘 쓴다

영상의 결론: 기술보다 '어떤 정보를 받으면 어떻게 처리하는가'가 중요하다

1

네 층을 차례로 붙인다

읽기, 판정, 대조, 실행 순서로 하나씩 붙이고 단계마다 결과를 확인한다.

2

판정 기준을 계속 쌓는다

애매한 사례를 labeling_guide에 더할수록 판정이 우리 회사에 맞아진다.

3

쓰기는 끝까지 사람이 승인한다

보내기와 발행은 action_policy에 적은 대로 사람이 마지막에 확인한다.

다음 과정: 같은 구조를 이메일, 슬랙, 구글 드라이브로 넓힌다